ScamGuard: AI-Based Online Scam and Fraud Detection System



PROJECT REPORT

Submitted by

Group Number: 56

BHAVYA GOYAL (1BM23CS063)

PRIYAM THAKUR (1BM23CS252)

SAMIR CHAUDHARY (1BM23CS294)

SANA SUBODH (1BM23CS296)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Dr. Kayarvizhy N

Professor, Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

Academic Year - 2026–2027

Certificate

This is to certify that the project report entitled **"ScamGuard: AI-Based Online Scam and Fraud Detection System"** is a bonafide work carried out by **BHAVYA GOYAL (USN: 1BM23CS063)**, **PRIYAM THAKUR (USN: 1BM23CS252)**, **SAMIR CHAUDHARY (USN: 1BM23CS294)** and **SANA SUBODH (USN: 1BM23CS296)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering, under my supervision during the academic year 2026–2027.

Guide Signature	HoD signature	Principal Signature
Dr. Kayarvizhy N	DR SREELATHA R PROFESSOR AND HOD	Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

DECLARATION

We hereby declare that the Major Project Phase-2 work entitled "ScamGuard: AI-Based Online Scam and Fraud Detection System" is a bonafide work carried out by us under the guidance of Dr. Kayarvizhy N, Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. To the best of our knowledge and belief, this project has not been submitted in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	BHAVYA GOYAL	1BM23CS063	
2	PRIYAM THAKUR	1BM23CS252	
3	SAMIR CHAUDHARY	1BM23CS294	
4	SANA SUBODH	1BM23CS296	

Place: Bengaluru
Date:

Certified that these candidates are students of the Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work titled "ScamGuard: AI-Based Online Scam and Fraud Detection System" as Major Project Phase-2 work in partial fulfillment of the requirements for the award of the B.E. degree by VTU. The work is original and is duly certified as such.

Guide Name: Dr. Kayarvizhy N
Date:
Signature

Acknowledgment

We express our sincere gratitude to our guide, Dr. Kayarvizhy N, Professor in the Department of Computer Science and Engineering, for her continuous guidance and encouragement. She kept asking us to back every claim with evidence, first pushed us to build data that matches what Indian phone users really receive, and later proposed the cross-domain generalization and adaptation study that became the advancement of this project.

We also extend our sincere thanks to Dr. Sreelatha R, Professor and Head of the Department of Computer Science and Engineering, for providing us with the necessary facilities, resources and a supportive academic environment that enabled us to carry out this project successfully. We are equally grateful to Dr. Bheemsha Arya, Principal of B.M.S. College of Engineering, and to all the faculty members of the Department of Computer Science and Engineering for their encouragement and support. The faculty who sat through our phase reviews asked hard questions about our evaluation, and this report is better for them.

The IMC 2025 smishing collection, the UCI SMS Spam Collection and the India Spam SMS collection were shared openly by their authors, and this work would not exist without them.

We would like to acknowledge our friends and classmates for their continuous encouragement and valuable suggestions. Many of the suspicious messages they and our families forwarded to us became test cases.

Finally, we express our deepest gratitude to our families for their unconditional support, constant encouragement and understanding throughout the duration of this project.

Abstract

In India a text message is still the cheapest way to reach someone's phone, and fraudsters use it heavily. Their messages copy bank KYC reminders, electricity disconnection warnings, parcel updates and job offers, and more and more of them are written in Hindi or in a Hindi–English mix. ScamGuard is an AI-based system that reads one such message, decides whether it is a scam or genuine, points to the phrases behind that decision and suggests the user's next step.

The detector learns from ScamGuard-IN-2026, a set of 11,399 Indian messages that we assembled from public smishing reports, real Indian SMS, 2025–26 advisory texts and synthetic messages that are used only for training. We did not trust the report labels blindly: after checking them, 19.5% of the reports we kept turned out to be genuine bank alerts, OTP messages or advertisements. Near-copies of the same campaign were grouped with MinHash so that a campaign never sits on both sides of the split. Logistic regression over word and character TF-IDF, joined with 31 readable intent patterns, scores 97.1% accuracy and 3.2% false alarms on 1,874 real messages from campaigns it never saw. Trained on the raw report labels instead, the same model raises false alarms to 34.0%.

Our guide proposed a further question as the advancement of the project: does a spam filter built on one region's messages still work on another's, and how much local data repairs it? We trained Multinomial Naive Bayes and Logistic Regression on the UK-based UCI SMS Spam Collection and ran them unchanged on an Indian SMS dataset. Logistic Regression dropped from a spam F1 of 0.911 to 0.678 and let through roughly half of the Indian spam. With only 10% of a labelled Indian pool (143 messages) added to training, the mean Indian F1 over three seeds rose from 0.681 to 0.964 while UK performance stayed the same. The improvement appeared in every seed, passed McNemar's test and remained when near-duplicate messages were kept apart. ScamGuard runs as a live web application (Next.js, FastAPI, PostgreSQL) with its own cross-domain page and a Streamlit demo, and the repository regenerates every figure in this report.

Keywords: scam detection, SMS spam, smishing, domain shift, domain adaptation, TF-IDF, logistic regression, Naive Bayes, label noise, data leakage

Contents

1 Introduction **5**

1.1 Overview 5
1.2 Motivation 5
1.3 Objective 6
1.4 Scope 6
1.5 Existing System 7
1.6 Proposed System 7

2 Literature Survey **9**

2.1 Overview 9
2.2 Detailed Survey 9
2.2.1 SMS Spam Collections and Classifiers 9
2.2.2 Smishing Detectors and Report-Based Corpora 9
2.2.3 Domain Adaptation and Evaluation Pitfalls 10
2.3 Research Gap Identification 12
2.4 Mapping of Objectives with Research Gap 13
2.5 Requirements for Evaluating SMS Scam Detectors 13

3 Software and Hardware Requirement Specification **15**

3.1 Functional Requirements 15
3.2 Non-functional Requirements 16
3.3 Hardware Requirements 16
3.4 Software Requirements 17
3.5 Cost Estimation 17

4 Design **19**

4.1 High Level Design 19
4.2 System Architecture 21
4.3 Use-Case Diagram 21
4.4 Class Diagram 22
4.5 Sequence Diagram 23

5 Implementation **25**
5.1 Implementation Overview . . . 25
5.1.1 Tools and Techniques . . . 27
5.2 Datasets and Label Audit . . . 28
5.2.1 Datasets of the cross-domain study . . . 29
5.2.2 Data fields . . . 30
5.3 Preprocessing and Feature Extraction . . . 31
5.3.1 Preprocessing . . . 31
5.3.2 Feature extraction . . . 31
5.4 Classifiers and Intent Fusion . . . 32
5.4.1 Logistic regression . . . 32
5.4.2 Multinomial Naive Bayes . . . 32
5.4.3 Intent evidence and fusion . . . 32
5.4.4 Inference procedure . . . 33
5.5 Campaign Grouping and Dataset Construction . . . 33
5.6 Cross-Domain Study . . . 34
5.6.1 Domain adaptation . . . 34
5.6.2 Experiment protocol . . . 34
5.7 Back End, Database and Front End . . . 35

6 Benchmarking and Results **38**
6.1 Evaluation Objective and Datasets . . . 38
6.2 Metrics . . . 38
6.3 Aggregate Results . . . 39
6.4 Cross-Domain Generalization and Adaptation . . . 42
6.4.1 Cross-domain generalization (RQ1 and RQ2) . . . 42
6.4.2 Lightweight domain adaptation (RQ4) . . . 43
6.4.3 Word versus character features (RQ3) . . . 44
6.4.4 Error analysis (RQ5) . . . 45
6.5 Interpretation Boundaries and Threats to Validity . . . 47
6.6 Application-Level Evaluation . . . 48

7 Conclusion and Future Enhancements **49**
7.1 Conclusion . . . 49
7.2 Future Enhancements . . . 49

Summary **51**

References **52**

Appendix A: Bibliography **52**

Appendix B: POs and PSOs Mapped **54**

Appendix C: Similarity report, AI Generated report **56**

1 Introduction

1.1 Overview

Most fraudulent text messages in India do not announce themselves. They read like the alerts people get every week: the bank wants a KYC update before midnight, the electricity board will cut the supply at 9.30 pm, a courier is holding a parcel until a ten-rupee fee is paid, or a traffic challan can be settled through an attached app. The fraudster borrows these formats precisely because people already trust them. That is what makes the problem hard for software, too. A genuine OTP message and a fake one share almost every word: OTP, account, KYC, link, today.

Automatic SMS filtering is an old research topic. The trouble is how it is usually tested. Most studies take one collection, very often the UCI SMS Spam Collection [4], shuffle it, split it at random and report accuracy above 95% [5, 6]. Such a figure only says how well the model handles messages similar to the ones it learned from. A filter used in India sees other brands, rupee amounts, shortened links and Hindi text, and nobody had checked how much of that 95% survives.

ScamGuard is our answer to both problems. A user pastes one message into a web page and gets back a verdict, a risk score and an explanation: which phrases are risky, which stage of a scam each phrase belongs to, how a real message from the same organisation would differ, and what to do next. Under the page sit a text classifier, a set of readable rules, an Indian dataset whose labels we checked ourselves, and a separate study of what happens to such classifiers when the data comes from a different country.

1.2 Motivation

Four things pushed us towards this project.

The first is scale. A 2025 study of over 64,000 public smishing reports traced most of the attributable sender numbers to Indian mobile operators [1]. India is not a side case; it is where much of the problem lives.

The second is the price of a false alarm. Real bank alerts and operator notices use the vocabulary of scams. Public smishing collections hold only messages that someone reported, and plenty of those reports turn out to be genuine. A checker that keeps flagging real bank messages trains its users to stop listening.

The third is explanation. Telling someone "spam" does not help them decide anything. They need to see the line that is the trap.

The fourth came from our guide, Dr Kayarvizhy N: domain shift. A model built on one country's messages may not carry over to another's. Theory says the error on a new domain is bounded by the error on the old one plus a term for how different the two are [21], and adding a small labelled sample from the new domain is the usual low-cost fix [22, 23]. For SMS spam, and for Indian messages in particular, neither the size of the drop nor the amount of local data needed had been measured.

1.3 Objective

The project sets out to:

1. Build a labelled collection of Indian scam and genuine SMS that covers current scam types, with labels that we verified instead of copying them from user reports.
2. Test lightweight text classifiers on it with a split that keeps near-identical messages from appearing in both training and test data.
3. Pair the best classifier with readable intent rules that explain each verdict phrase by phrase, and check honestly whether those rules also raise accuracy.
4. Measure how well standard spam classifiers (Naive Bayes and Logistic Regression over word and character TF-IDF) trained on the UCI SMS Spam Collection work on Indian SMS, and study how much adding 10%, 20% and 40% labelled Indian data helps.
5. Put the detector and the cross-domain models into a working, deployed web application, together with safety tools, history and analytics.
6. Keep every reported number reproducible from the project repository.

1.4 Scope

ScamGuard looks at one message at a time, in English, Hindi written in Devanagari, or Hindi typed in Latin letters. The main detector answers *scam* or *legitimate*. We call a message a scam if it uses deception to get money, a secret such as an OTP or PIN, personal details, an app install or a call to a private number; an honest advertisement counts as legitimate. The cross-domain study keeps the labels of its two public datasets, *spam* and *ham*, where spam also covers unwanted promotions. Besides plain text, the application takes URLs, e-mails, screenshots, PDFs and QR codes, turning them into text or passing them to separate checks.

Some things are deliberately left out. ScamGuard does not read messages on the phone

automatically, the classifier sees neither the sender nor earlier messages in the thread, and Indian languages other than Hindi are not handled. The cross-domain study compares two datasets only, so it makes no general claim about every possible domain. The risk score orders messages by risk but is not a calibrated probability, and the verdict is guidance, not proof that a crime took place.

1.5 Existing System

What exists today falls into three groups.

- **Measures taken by operators and the regulator.** Operators screen bulk traffic and register commercial senders, and anyone can report a message to the 1909 DND service or the cyber-crime portal. These steps work on senders and volumes. They do nothing for a person staring at a message that has already arrived.
- **Keyword and rule filters.** Many phone apps warn on words like "lottery" or "KYC". The logic is easy to follow, but the same words appear in real messages, and a scam that avoids them goes through.
- **Machine-learning filters from research.** Papers on Naive Bayes, SVM, CNN and LSTM report high accuracy, yet nearly always on promotional spam, with random splits inside one dataset, with no genuine Indian messages and with no test on another region's data [5, 6, 9]. The newer corpora built from user reports contain reported messages only [10, 1].

1.6 Proposed System

Our system has four pieces.

1. **The ScamGuard-IN-2026 dataset.** It mixes real reported Indian scams, real genuine Indian messages, texts from 2025–26 government advisories and synthetic messages for the newest scam types. Every group of reported messages was checked, and the split is made by scam campaign.
2. **The detector.** Logistic regression on word and character TF-IDF, joined with readable intent rules and guards for genuine messages, so each verdict comes with its reasons.
3. **The cross-domain study.** Naive Bayes and Logistic Regression are trained on the UCI collection and tested on Indian SMS, first as they are and then after adding 10%, 20% and 40% of a labelled Indian pool. Everything runs with three seeds and is backed by paired significance tests, a near-duplicate check and an error analysis done by hand.

4. **The application.** A deployed website offering a quick check, a full analysis with Scam X-Ray and TrustLens, an AI copilot, safety tools, history, analytics and a cross-domain page where the baseline and adapted models can be compared; the study also has its own Streamlit app.

2 Literature Survey

2.1 Overview

We studied nineteen publications. They cover public SMS spam collections, smishing detectors, large studies built from user reports, the theory and practice of domain adaptation, and methods for catching label noise and train–test leakage. For every entry we opened the publisher page, DOI record or arXiv listing to confirm the details, and the figures we quote are the authors' own.

2.2 Detailed Survey

2.2.1 SMS Spam Collections and Classifiers

The SMS Spam Collection. In 2011 Almeida, Gómez Hidalgo and Yamakami published a set of 5,574 English text messages, 747 of them spam, collected mainly in the UK [4]. Two years later they tried a large number of classifiers on it; a linear SVM came out on top with 97.64% accuracy and only 0.18% of good messages blocked [5]. The spam in this set is mostly prize draws and ringtone offers, which look little like an Indian KYC or electricity scam today.

Deep models on the same data. Roy, Singh and Banerjee reached as much as 99.44% accuracy with CNN and LSTM networks [6]. Ahmadi *et al.* tested six classifiers on TF-IDF and bag-of-words features and got between 0.91 and 0.95 [13]. Labonne and Moran found that a fine-tuned Flan-T5 model outperforms classical methods on e-mail spam even with a small training set [12]. Every one of these studies trains and tests inside a single dataset.

2.2.2 Smishing Detectors and Report-Based Corpora

Smishing detectors. SmiDCA, by Sonowal and Kuppusamy, picks content features for an anti-smishing classifier [8]. Mishra and Soni looked at both the text and what the embedded link does, and reported 96.29% accuracy [7]. Goel *et al.* normalised slang and trained Naive Bayes on 362 smishing texts, ending at 96.2% accuracy with 3.87% false positives [9].

Corpora built from reports. Timko and Rahman released 1,062 Smishtank submissions that they had validated [10]. Agarwal *et al.* put out 33,869 reported messages in 66 languages, labelled by GPT-4o, and point out that nobody verified the reports [1]. In

a separate paper the same authors found that 35.12% of unique UK operator reports were ordinary spam and not scams [2]. For Mozambique, MOZ-Smishing offers both real and fraudulent mobile-money messages [11].

2.2.3 Domain Adaptation and Evaluation Pitfalls

Domain shift and adaptation. Ben-David *et al.* proved that a classifier's error on a new domain is at most its error on the training domain plus a measure of how far apart the two domains are [21]. Pan and Yang's survey of transfer learning includes the case where a little labelled data from the new domain is available [22]. Daumé III showed that very simple ways of mixing source and target data are hard to beat in supervised adaptation for language tasks [23]. Not one of the SMS papers above checks a model on a second collection.

Reviews and method papers. Pritom *et al.* went through 119 smishing papers and describe the datasets as "often limited in scale, diversity, and temporal coverage" [3]. Northcutt, Jiang and Chuang use cross-validated disagreement to find wrongly labelled examples [14]. Lee *et al.* show how near-duplicates inflate test scores [15], and Kapoor and Narayanan rank duplication between train and test sets among the most frequent forms of leakage [16]. Near-duplicates are usually found with MinHash [17].

The works are summarised in Table 2.1.

Table 2.1: Summary of related work

No.	Work	Approach	Reported result	Why it is not enough here
1	Almeida *et al.*, 2011 [4]	Public SMS spam collection with baselines	5,574 labelled texts	UK promotions; nothing Indian
2	Almeida *et al.*, 2013 [5]	Over 15 classifiers and tokenisers compared	SVM at 97.64% accuracy	One corpus, one domain
3	Roy *et al.*, 2020 [6]	CNN and LSTM networks	99.44% accuracy	Never tested on other data
4	Sonowal, Kuppusamy, 2018 [8]	Content feature selection with ML	Anti-smishing classifier	Small English data
5	Mishra, Soni, 2020 [7]	Text plus link behaviour	96.29% accuracy	Weak on scams with no link
6	Goel *et al.*, 2024 [9]	Slang clean-up and Naive Bayes	96.2% accuracy, 3.87% FPR	Just 362 smishing texts

No.	Work	Approach	Reported result	Why it is not enough here
7	Timko, Rahman, 2024 [10]	Collection of validated reports	1,062 reports	No genuine messages
8	Agarwal *et al.*, 2025 [1]	Reports labelled by GPT-4o	33,869 texts	Unverified; no genuine messages
9	Agarwal *et al.*, 2026 [2]	Study of operator reports	35.12% were plain spam	UK only; no detector
10	Ali *et al.*, 2025 [11]	In-context learning with LLMs	Mobile-money benchmark	Built for Mozambique
11	Labonne, Moran, 2023 [12]	Fine-tuned Flan-T5	Ahead of classical models	E-mail rather than SMS
12	Ahmadi *et al.*, 2025 [13]	Six classifiers on TF-IDF	Accuracy 0.91–0.95	One dataset
13	Pritom *et al.*, 2026 [3]	Systematic review	119 papers, 12 datasets	Points out the data gap
14	Ben-David *et al.*, 2010 [21]	Theory of learning across domains	Bound on target error	Theory, no SMS data
15	Pan, Yang, 2010 [22]	Survey of transfer learning	Map of settings	Not about SMS
16	Daumé III, 2007 [23]	Feature augmentation	Simple, strong baseline	Other language tasks
17	Northcutt *et al.*, 2021 [14]	Confident learning	Locates label errors	Generic method
18	Lee *et al.*, 2022 [15]	Removing near-duplicates	Overlap inflates scores	Studied on LM corpora
19	Kapoor, Narayanan, 2023 [16]	Survey of leakage	Duplication is frequent	Generic method

2.3 Research Gap Identification

Reading these works side by side, we found five gaps.

1. **There is no checked class of genuine Indian messages.** The classical sets are not Indian, and the report-based ones hold reported messages only.
2. **Report labels are trusted as they are,** even though the people who collected them say the reports were never verified.
3. **Testing ignores campaigns.** With a random split, copies of the same scam template land in both training and test data and push the score up.
4. **Rules are not tested on independent data.** Hybrid systems add rules but judge them on examples picked by the same developers.
5. **SMS spam models are not tested across domains.** Published classifiers are trained and scored inside one collection, so nobody knows how far they fall on another country's messages or how much local labelled data brings them back.

From these gaps we framed the problem as follows:

Design, build and evaluate an AI-based system that tells Indian scam SMS from genuine ones with few false alarms, using checked labels and a split free of leakage; measure how standard spam classifiers trained on one SMS domain cope with Indian SMS and how much labelled Indian data adaptation requires; and offer both through a deployed web application that explains every verdict.

2.4 Mapping of Objectives with Research Gap

Table 2.2: Mapping of objectives to research gaps

Objective	**Gap addressed**
1. Indian dataset with verified labels	Gaps 1 and 2: a genuine Indian class and an audit of report labels
2. Testing without leakage	Gap 3: split by campaign
3. Classifier plus readable rules, with the rules measured	Gap 4: rules judged on independent real messages
4. Cross-domain testing and adaptation	Gap 5: UCI-to-India test with 10/20/40% adaptation
5. Deployed application	Puts the findings to practical use
6. Reproducibility	Lets others verify the work on all five gaps

2.5 Requirements for Evaluating SMS Scam Detectors

The surveyed work and the five gaps above lead to a short list of requirements for a scam detector that is meant to be used and not only scored. These requirements come from the nature of the task, and they shaped the design that follows.

Labels must be checked before they are trusted. Reports of scams contain genuine messages: 35.12% of the unique UK operator reports in one study were plain spam [2], and 19.5% of the Indian reports we kept turned out to be genuine. A model trained on such labels learns to flag real bank alerts, so an audit has to come before training (Section 5.2).

Evaluation must keep near-identical messages apart. One campaign is sent with small edits to many people, so a random split puts copies of the same message on both sides and inflates the score. Messages therefore have to be grouped into campaigns before the split is made (Section 5.5).

A model must meet a second domain before it is trusted. A filter that looks excellent on one collection may miss half of the spam in another, and the cost of repairing it with local labels should be measured and not assumed (Section 5.6).

Every verdict must be explained. A user who is only told “spam” cannot decide what to do, so the system has to show which phrases make up the trap and what a genuine message would look like (Section 5.4).

Finally, the service has to stay cheap and fast. A small linear model with readable

weights meets this need on a free cloud server, which a large neural model would not.

3 Software and Hardware Requirement Specification

3.1 Functional Requirements

Functional requirements describe what the system must be able to do.

1. **Quick check.** Without creating an account, a visitor pastes a message on the home page and gets a verdict, a risk score, the trap phrases highlighted and a short explanation.
2. **Accounts.** People can register and sign in (with a captcha) or try a demo account, and they can edit, export or delete their own data.
3. **Full analysis.** Once signed in, a user can submit text, a URL, an e-mail, a screenshot, a PDF or a QR code. The result shows the verdict, the classifier's probability, the final risk score, the Scam X-Ray stages, the TrustLens report and suggested actions.
4. **Explanation.** Every result lists the phrases that matched, the scam stage each belongs to (hook, pressure, request, cash-out), the evidence on both sides and the way a genuine message would look.
5. **Cross-domain page.** A visitor picks the baseline model (UCI only) or the adapted one (UCI plus 40% of the Indian pool), chooses Naive Bayes or Logistic Regression and classifies a message. The page returns the label, the spam probability, the model version and the features that drove the decision, next to the study's measured results.
6. **AI copilot.** Follow-up questions in English or Hindi go to a hosted language model. Each visitor has a limit, there is a daily cap, and fixed fallback answers cover outages.
7. **Safety tools.** A QR code check, a "before you pay" check for UPI requests, a check for payment screenshots, a file check that catches disguised Android apps, and a chat timeline.
8. **History, cases and reports.** Past scans can be searched and grouped into cases; users can leave feedback, download a PDF report and post a scam to the community view.
9. **Analytics and administration.** Users see their totals and trends. Administrators also see the registered model versions and can run a robustness test.

3.2 Non-functional Requirements

These requirements describe the qualities the system must have.

- **Performance.** On a two-core server the classifier needs about 2 ms per message, about 4 ms with the intent layer. A complete web request takes roughly 1–2 seconds.
- **Reliability.** Models are read from versioned files once, when the server starts. The back end keeps itself from sleeping on the free tier, and the front end retries short-lived proxy errors.
- **Security.** Passwords are stored as bcrypt hashes; access relies on signed JWT tokens whose refresh tokens rotate; every endpoint has a rate limit and a size limit on the request body; fetching outside URLs is protected against SSRF; and security headers are set.
- **Privacy.** Nothing typed into the quick check or the cross-domain page is saved, and contributed messages are masked before they are stored.
- **Usability.** The interface works in English and Hindi, has an elder mode, can be driven from the keyboard and fits phone screens.
- **Maintainability and reproducibility.** Training and serving call the same preprocessing code. Continuous integration runs 836 back-end and ML tests and 14 front-end tests, and a single notebook plus a single script regenerate every number we report.

3.3 Hardware Requirements

ScamGuard needs no GPU. Table 3.1 lists what each part of the system needs.

Table 3.1: Hardware requirements

Component	**Requirement**
Development machine	An ordinary laptop with 8 GB RAM and a recent 64-bit CPU; a GPU is not required
Training	On a laptop CPU, under 3 seconds for the production model and around a minute for the whole cross-domain notebook
Server	A single small cloud instance; 512 MB RAM is enough
Client	Any web browser on a phone or a computer

3.4 Software Requirements

Table 3.2 lists the software used for each layer of the system.

Table 3.2: Software requirements

Layer	**Software**
Language	Python 3.10+ (tested on 3.13), TypeScript 5.5
Machine learning	scikit-learn 1.7.2, NumPy, pandas, SciPy, joblib, datasketch, ftfy, matplotlib
Notebook and study app	Jupyter or Google Colab, Streamlit
Back end	FastAPI 0.115, Uvicorn, SQLAlchemy 2.0, Alembic, Pydantic 2, SlowAPI, python-jose, passlib, ReportLab, Tesseract OCR
Front end	Next.js 14.2, React 18.3, Tailwind CSS 3.4, Recharts, Vitest
Database	PostgreSQL (Neon) in production, SQLite locally
Deployment	Vercel (front end), Render (back end), GitHub Actions (continuous integration)

3.5 Cost Estimation

ScamGuard was built as a student project on free tiers and open-source software, so the cost of building and running it at the present scale is zero.

Development software: Python, scikit-learn, FastAPI, Next.js, Streamlit, Jupyter and VS Code are open source and cost nothing.

Data: All three public datasets (the IMC 2025 smishing reports, the UCI SMS Spam Collection and the India Spam SMS collection) were used under their open licences and cost nothing.

Hosting: The front end runs on the free Hobby plan of Vercel, the back end on the free plan of Render and the database on the free plan of Neon.

AI copilot: The copilot calls a hosted language model within its free quota. A daily cap on the whole service and a limit for each visitor stop it from running past that quota, and fixed fallback answers cover outages.

Total estimated cost: zero for development and hosting. These free tiers have memory and traffic limits (the back end has 512 MB of memory), so a service with sustained heavy traffic would need paid hosting; we have not measured that cost.

4 Design

4.1 High Level Design

ScamGuard follows the usual three-tier layout. At the top, the **presentation tier** is a Next.js front end on Vercel, and every API call goes out through its own origin. In the middle, the **application tier** is a FastAPI back end on Render; one process runs the detector, the intent rules, the cross-domain models, sign-in and the safety tools. At the bottom, the **data tier** is a PostgreSQL database on Neon holding users, scans, cases and reports. Training happens offline. The trained models are saved as versioned files, which the back end loads a single time when it starts. Figure 4.1 shows how the pieces connect.

Table 4.1 breaks ScamGuard into modules and gives each one's job and the way other parts talk to it.

Table 4.1: Modules of ScamGuard and their interfaces

Module	**Responsibility**	**Interface**
Dataset builder	Gathers the sources, audits labels, groups campaigns and splits the data	Python script that writes one CSV file
Training pipeline	Trains and compares models, then registers the chosen one	Model registry (vectorizer and model files)
Detector	Cleans, classifies, applies rules, fuses scores and explains	Python function predict(text)
Cross-domain study	Audits and cleans the UCI and Indian data, runs the seven experiments over three seeds, saves results and models	Jupyter notebook producing CSV tables, figures, results.json and joblib models
Cross-domain service	Serves the baseline and adapted models, with explanations	POST /public/cross-domain/classify, GET /public/cross-domain/results
Back end (API)	Sign-in, rate limits, storage, reports and safety tools	REST over HTTPS, JSON
Front end	Quick check, analysis, dashboard, tools, cross-domain page, Hindi interface	Browser; calls the REST API
Streamlit app	Separate demo of the study	Loads the saved models and never retrains them
Copilot	Answers follow-up questions	REST endpoint calling an outside LLM API, with limits

4.2 System Architecture

The system has an offline part and an online part, shown in Figure 4.1. The shaded blocks run offline: the cross-domain notebook, the training on ScamGuard-IN-2026 and the model registry. Training writes versioned model files to the registry, and the back end loads them a single time when it starts. The online path follows each request through the web app, the FastAPI back end, the cleaner and tokenizer, the TF-IDF and logistic-regression classifier, the intent rules, the fusion step and the explanation layer, and ends by saving the scan in PostgreSQL. The AI copilot and the known-pattern retrieval (52 entries) feed the explanation layer, and the cross-domain page calls the two stored model pairs directly.

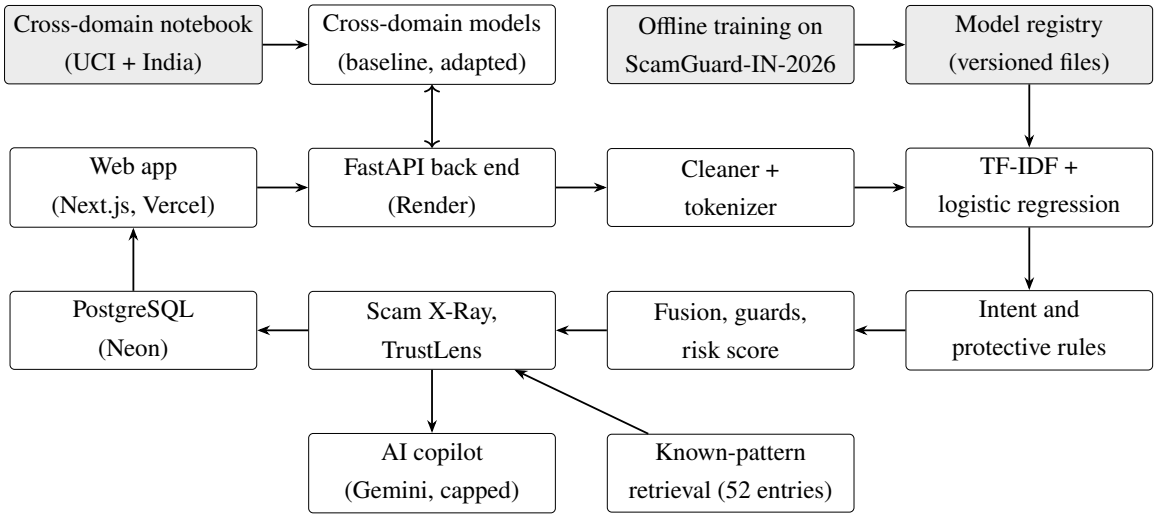


Figure 4.1: Architecture of the deployed system. Shaded parts run offline.

4.3 Use-Case Diagram

The use-case diagram in Figure 4.2 shows how the three kinds of user interact with ScamGuard. A visitor can run a quick check and use the cross-domain page without an account. A registered user can do everything a visitor can and also run a full analysis, read the Scam X-Ray and TrustLens reports, use the safety tools, keep history and cases, and ask the AI copilot, which calls an outside hosted language model. An administrator also sees the registered model versions and can run a robustness test.

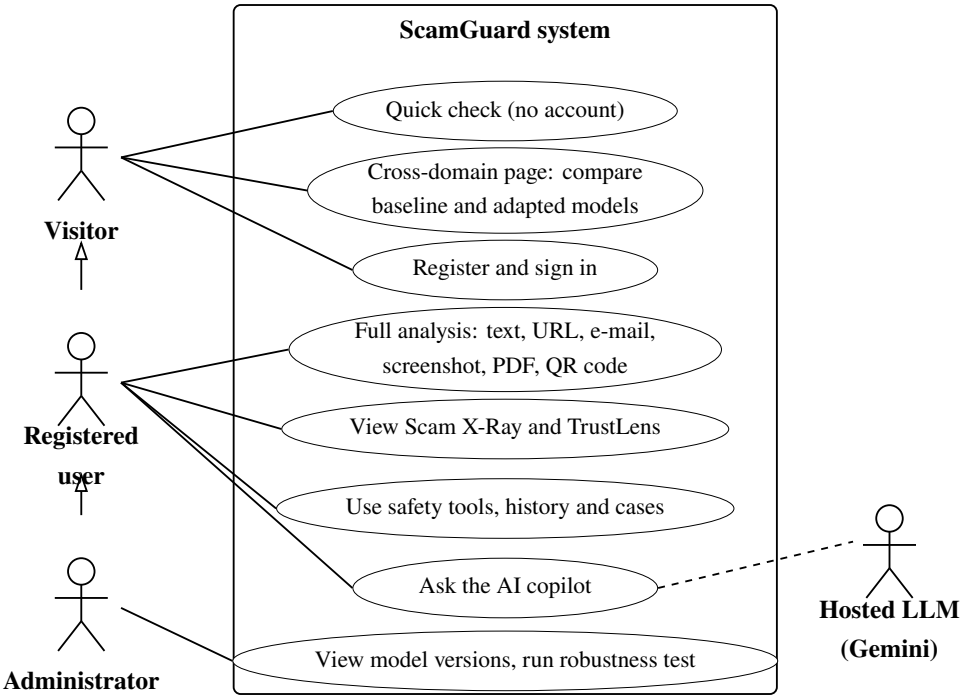


Figure 4.2: Use-case diagram of ScamGuard

4.4 Class Diagram

The class diagrams outline the object-oriented structure of the back end. Figure 4.3 shows the application layer: MessageService receives a scan request, stores it through two repositories and calls PredictionService, while AuthService handles sign-in and ExtractionService turns URLs, e-mails, images, PDFs and QR codes into text. Figure 4.4 shows the detection components: PredictionService combines the InferenceEngine, the ThreatScorer, the ConfidenceCalculator and the PredictionExplainer. The engine loads the production model through the ModelRegistry and uses the same preprocessing pipeline and TF-IDF extractor that training used.

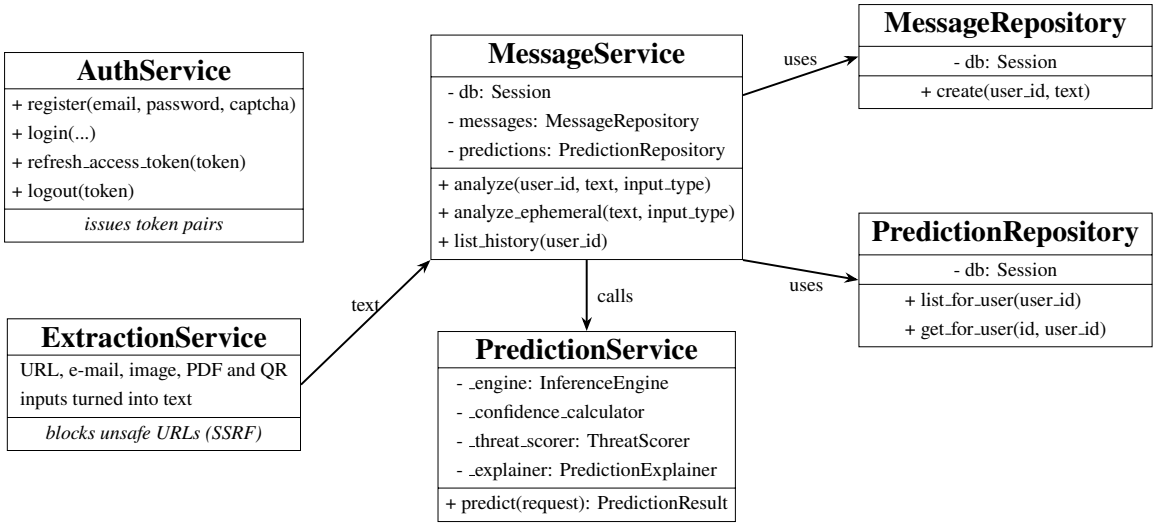


Figure 4.3: Class diagram of the application layer

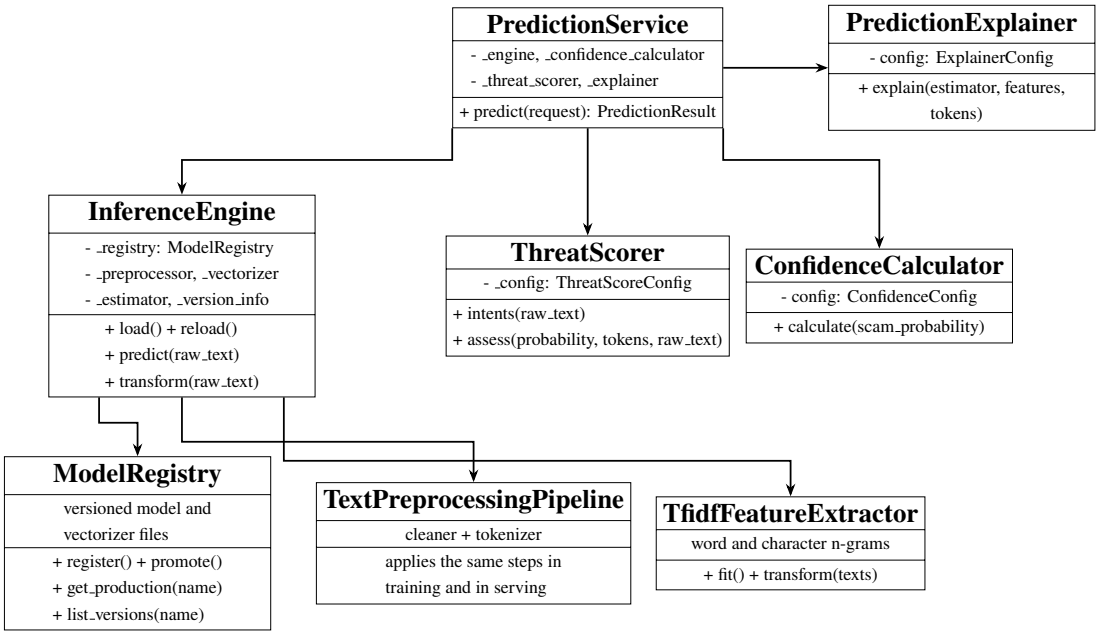


Figure 4.4: Class diagram of the detection components

4.5 Sequence Diagram

The browser and the back end exchange JSON over HTTPS. The sequence diagram in Figure 4.5 traces a single analysis from request to reply.

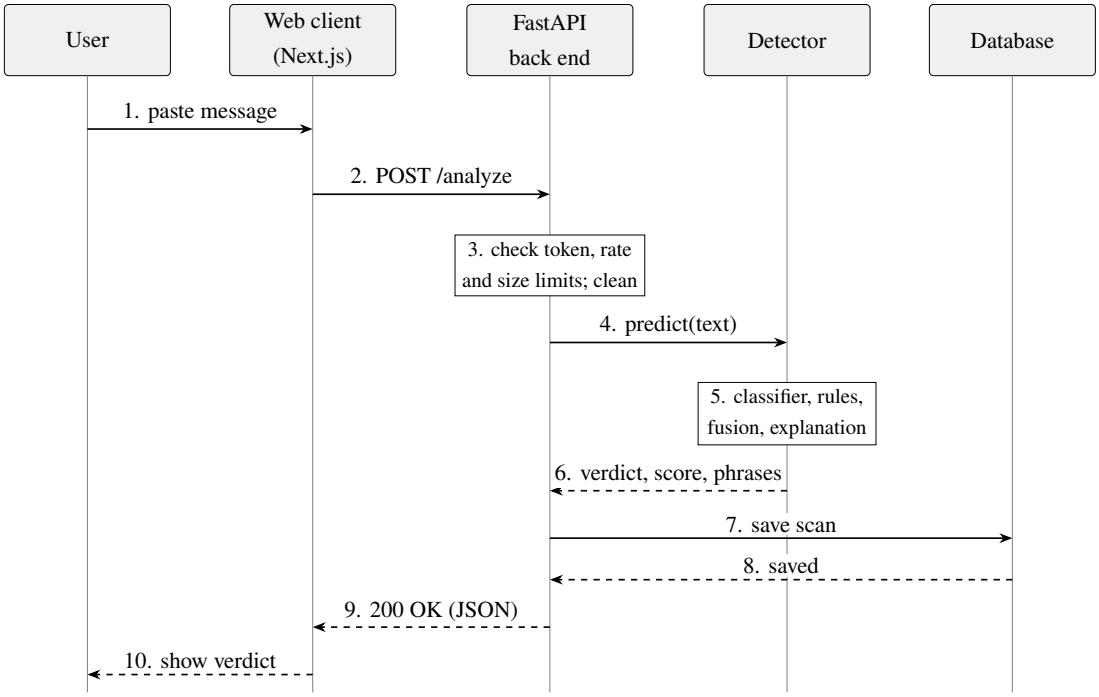


Figure 4.5: Sequence diagram of one message analysis

- **Endpoints.** The three classification routes are listed in Table 4.2. Further routes cover sign-in, history, cases, analytics, the copilot and the safety tools.
- **Authentication.** Short-lived signed access tokens paired with refresh tokens; passwords kept as bcrypt hashes.
- **Input limits.** A message may be up to 4,000 characters long, or 2,000 on the public endpoints.

Table 4.2: Classification endpoints and their rate limits

Route (POST)	**Used by**	**Requests/min**
/api/v1/public/quick-check	Visitors (no account)	10
/api/v1/messages/analyze	Signed-in users	30
/api/v1/public/cross-domain/classify	Cross-domain page	30

5 Implementation

5.1 Implementation Overview

ScamGuard was built in two parts that share one preprocessing code base: an offline part that builds the dataset and trains the models, and an online part that serves them. Figure 5.1 shows the whole flow from data collection to deployment, and the sections below describe each stage. Section 5.1 lists the tools and techniques used.

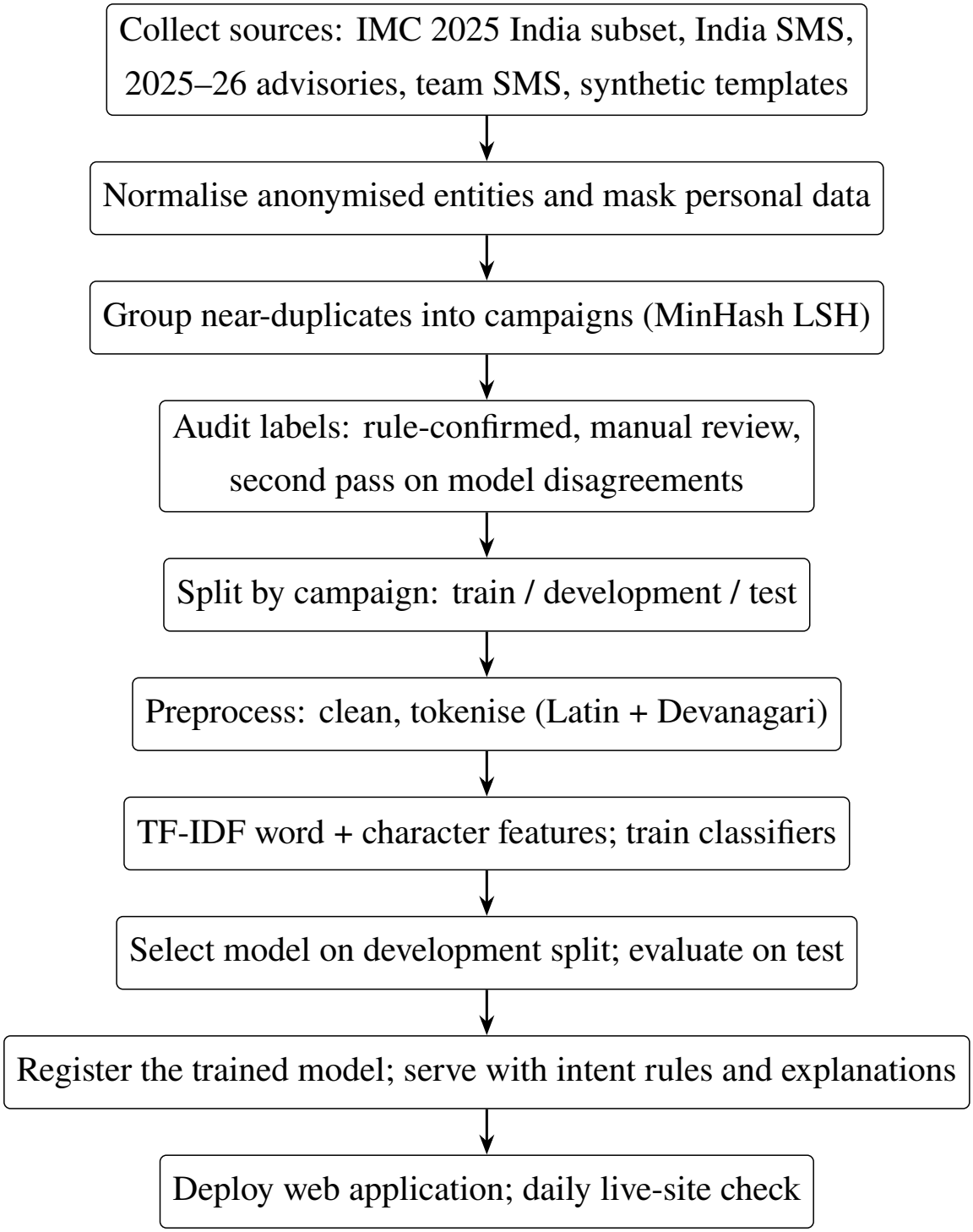


Figure 5.1: Flow of the methodology from data collection to deployment

The cross-domain study follows its own pipeline, shown in Figure 5.2.

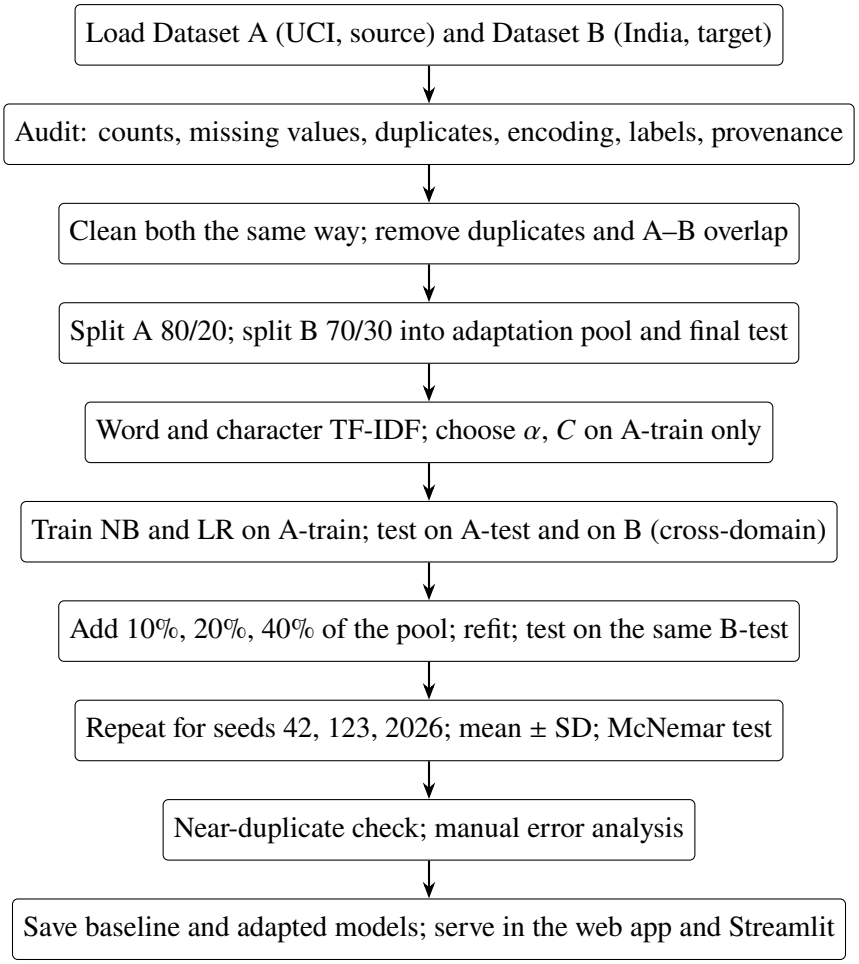


Figure 5.2: Flow of the cross-domain study

5.1.1 Tools and Techniques

- **TF-IDF** [18] gives a term a high weight when it is frequent in the message but uncommon in the collection. In the main detector we join word 1–2-grams with character 2–5-grams. The cross-domain study keeps them apart: word 1–2-grams on one side, character 3–5-grams on the other.
- **Logistic regression** is a linear model, so its coefficients can be read directly: each one says how strongly a word pulls a message towards scam or towards genuine.
- **Multinomial Naive Bayes** learns how often each term appears in each class and then applies Bayes' rule. It serves as the classical baseline in the cross-domain study.
- **MinHash and locality-sensitive hashing** [17] find messages that are nearly identical, which lets us group them into campaigns.
- **Confident learning** [14] tells us which labels to re-read: the ones a cross-validated model strongly disagrees with.
- **Intent rules written as regular expressions** spot requests for an OTP, "pay to receive"

offers, remote-access apps and legal threats, as well as reassuring signals such as "do not share your OTP".

- **Supervised domain adaptation** here simply means adding a labelled sample of target messages to the source training data [23, 22].
- **Bootstrap intervals, mean ± SD across seeds and McNemar's test** [24, 25] show how certain each result is and whether two models differ on the same messages.
- **Software:** scikit-learn [19] for the models, ftfy to fix broken text encodings, and FastAPI, Next.js and Streamlit for the applications.

5.2 Datasets and Label Audit

Indian scams and genuine Indian service messages use largely the same words, and public smishing data offers only reported messages whose labels nobody checked. We therefore assembled our own corpus, ScamGuard-IN-2026, with real genuine Indian messages, verified labels and the scam types seen in 2025–26. Its sources are listed in Table 5.1.

Table 5.1: Sources of ScamGuard-IN-2026

Source	Licence	Genuine	Scam	Use
R1: IMC 2025 smishing collection, India subset, audited [1]	CC BY 4.0	963	3,987	train, development, test
R2: India Spam SMS collection, audited [20]	MIT	1,694	6	train, development, test
R3: 2025–26 advisory and news texts	Quoted with source	1	11	test only
S: Synthetic templates	Project	2,950	1,787	training only
Total		**5,608**	**5,791**	**11,399 messages**

Checking the labels. Everything in the IMC 2025 collection is something a user *reported*, which is not the same as a confirmed scam. We took 2,922 decisions over the groups of reported near-duplicates. In 1,253 cases a rule settled it: the message carried a strong fraud cue (an account threat with a KYC instruction, a prize that needs a fee, a request for an OTP) and nothing that suggested a genuine sender; a spot check of 40 of these found no mistake. The remaining 1,669 we read one at a time and marked 583 as scam, 867 as genuine and 219 as unclear; the unclear ones were dropped. As a second pass, a cross-validated model pointed to 44 rows whose labels it strongly disagreed with.

Re-reading them led to 19 corrections and 7 removals. In the end, 963 of the 4,950 IMC rows we kept (19.5%) are genuine messages.

Campaigns and splits. Two messages fall into one campaign if the Jaccard similarity of their word 3-shingles, estimated with MinHash, reaches 0.6. This turns the 6,662 real messages into 4,085 campaigns. A hash of each campaign's first message then sends the whole campaign to training (60%), development (15%) or test (25%); Table 5.2 gives the counts.

Table 5.2: ScamGuard-IN-2026 splits

Split	Genuine	Scam	Content
Train	4,581	4,064	real and synthetic
Development	361	507	real only
Test	665	1,209	real only, unseen campaigns
2026 test	1	11	advisories

5.2.1 Datasets of the cross-domain study

Our guide named both datasets. **Dataset A, the source,** is the UCI SMS Spam Collection [4], released under CC BY 4.0. It holds 5,574 messages from the UK (4,827 ham, 747 spam) and is described in the paper that introduced it. **Dataset B, the target,** is the India Spam SMS Classification set [20], published in 2024 under the MIT licence. Its 2,267 rows have two fields, Msg and Label, with 1,522 ham and 745 spam. People submitted the messages through an online form, and the repository says nothing about how labels were assigned.

What the audit found. Dataset B contained one empty message and 204 exact repeats. In 176 rows the emoji and punctuation had been saved as garbled bytes, and in 40 rows the Devanagari text had been turned into question marks that cannot be recovered. We found no message carrying two different labels. Reading ten random messages from each class, the labels looked reasonable: the spam is mostly operator, bank and shop promotions with a link, and the ham is mostly short personal or group-chat text. Only one message, "ok", appeared in both datasets, and we removed it from B.

Cleaning. Both datasets went through identical steps. Labels became spam = 1 and ham = 0, unusable rows were dropped, garbled encodings were fixed with ftfy, everything was lowercased, runs of spaces were collapsed and exact repeats were removed. We left URLs, numbers and punctuation in place, since they are often what gives spam away. Table 5.3 shows the counts after each dataset was cleaned; Figures 5.3 and 5.4 show the class balance and message lengths.

Table 5.3: Cross-domain datasets before and after cleaning

Dataset	Raw	Clean	Ham	Spam	Spam %
A: UCI SMS Spam Collection	5,574	5,158	4,517	641	12.4
B: India Spam SMS	2,267	2,050	1,314	736	35.9

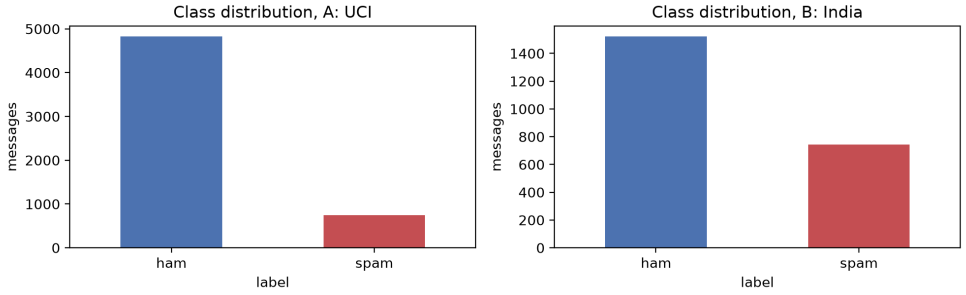


Figure 5.3: Class distribution of Dataset A (UCI) and Dataset B (India) before cleaning

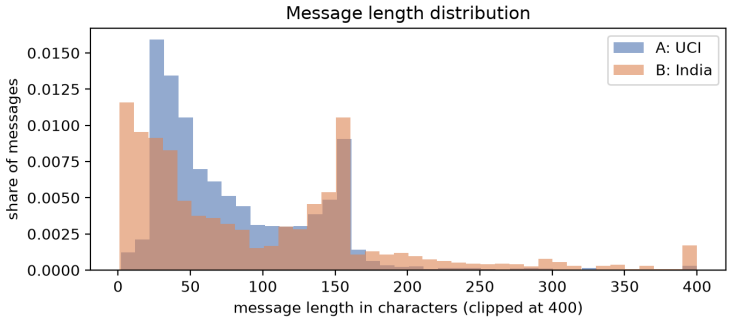


Figure 5.4: Message length distribution of the two datasets

5.2.2 Data fields

Each row of **ScamGuard-IN-2026** has seven fields: text; label, which is scam or legitimate; source; subtype; is_synthetic; campaign, naming the near-duplicate group; and split, which is train, development, test or 2026 test.

After cleaning, the **cross-domain datasets** keep three fields: message, label (spam = 1, ham = 0) and domain (UCI or India). Alongside them the study saves every result table as a CSV file, the error sheet with its hand-filled manual_category column, a results.json summary and four joblib models, namely Naive Bayes and Logistic Regression, each as a baseline and as an adapted version.

5.3 Preprocessing and Feature Extraction

Given a message x, the detector produces a score $s(x) \in [0, 1]$ and decides

$$\hat{y} = \begin{cases} 1 \text{ (scam/spam)} & \text{if } s(x) \geq 0.5 \\ 0 \text{ (legitimate/ham)} & \text{otherwise.} \end{cases} \tag{5.1}$$

5.3.1 Preprocessing

The main detector's cleaner swaps e-mail addresses, URLs (bare ones like bit.ly/x included) and phone numbers for placeholder tokens. It then lowercases the text, strips punctuation without damaging Devanagari vowel signs, and turns any other number into a number token. The tokenizer keeps Latin and Devanagari words, drops English stop words and trims common suffixes. Take *"Dear customer, your SBI account will be blocked today. Update your KYC at http://sbi-kyc.xyz and share the OTP"*: it comes out as dear customer sbi account will block today update kyc __url__ share otp. Training and the live service call the very same code. One side effect is worth stating: since punctuation and digits disappear before features are built, spellings such as *K.Y.C* or *bl0cked* lose their keyword, and such messages have to be caught by their other words or by the intent rules. The cross-domain study cleans far less, as the guide asked. It only lowercases and tidies whitespace, so URLs, numbers and punctuation stay; the word vectorizer splits on punctuation, while the character vectorizer sees it.

5.3.2 Feature extraction

For a term t in message d, with N training messages and document frequency $\mathrm{df}(t)$, the TF-IDF weight is

$$w_{t,d} = g(\mathrm{tf}_{t,d}) \cdot \left(\ln \frac{1+N}{1+\mathrm{df}(t)} + 1 \right), \tag{5.2}$$

after which each message vector is scaled to unit length. For the main detector g is sublinear, $g(\mathrm{tf}) = 1 + \ln \mathrm{tf}$, and the vocabulary has word 1–2-grams (at most 30,000) plus character 2–5-grams (at most 60,000); pruning leaves 35,148 features. The cross-domain study uses plain counts, $g(\mathrm{tf}) = \mathrm{tf}$, and either word 1–2-grams (at most 20,000) or character 3–5-grams taken inside word boundaries. No vectorizer ever sees anything but training data.

A small example. Take three messages: "update kyc now", "kyc update otp" and "order delivered today", so $N = 3$. In the first one, *update* and *kyc* each occur in two messages ($\mathrm{df} = 2$) and *now* in one, giving IDF values of $\ln(4/3) + 1 = 1.288$ and $\ln 2 + 1 = 1.693$. Normalised, the vector is $(0.518, 0.518, 0.681)$, so the word that is rarest in the collection

gets the most weight.

5.4 Classifiers and Intent Fusion

5.4.1 Logistic regression

Logistic regression estimates

$$p(x) = \sigma\big(\mathbf{w}^\top \phi(x) + b\big) = \frac{1}{1 + e^{-(\mathbf{w}^\top \phi(x) + b)}}, \tag{5.3}$$

by minimising the class-weighted, ℓ_2-regularised log-loss

$$\min_{\mathbf{w}, b} \; \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i} \Big[- y_i \ln p(x_i) - (1 - y_i) \ln\big(1 - p(x_i)\big) \Big], \tag{5.4}$$

where the class weights are $\alpha_c = n/(2 n_c)$. We set $C = 16$ for the main detector. In the cross-domain study C came from cross-validation on the source training data: 100 for word features and 10 for character features. The trained main model has bias $b = -4.82$. Among its strongest weights, *prize* (+2.98), *will block* (+2.87) and *kyc* (+2.69) push towards scam, while *branch* (−2.22) and *not share* (−1.85) push towards genuine. The SBI message from the preprocessing example gets $z = \mathbf{w}^\top \phi(x) + b = 8.99$, so $p = 0.9999$. A real OTP alert, “482913 is your OTP for Rs 2,499 at Amazon. Do not share it with anyone”, gets $z = -5.53$ and $p = 0.004$.

5.4.2 Multinomial Naive Bayes

Naive Bayes predicts the class with the larger posterior,

$$\hat{c} = \arg\max_{c \in \{0,1\}} \Big[\ln P(c) + \sum_{t} x_t \ln P(t \mid c) \Big], \qquad P(t \mid c) = \frac{N_{tc} + \alpha}{N_c + \alpha |V|}, \tag{5.5}$$

Here x_t is the TF-IDF value of term t, N_{tc} is the summed weight of t in class c, N_c is the summed weight of all terms in c, $|V|$ is the vocabulary size and α is the smoothing constant. Cross-validation on source training data picked $\alpha = 0.03$. We also report the library default, $\alpha = 1$, because the difference turns out to matter a great deal.

5.4.3 Intent evidence and fusion

The rule layer has 31 patterns for 26 scam intents, each with a weight w_i that we set by hand, and ten protective signals with weights v_j. Let $E^+ = \sum_{i \in I(x)} w_i$, halved if only weak

context cues fire, and $E^- = \sum_{j \in P(x)} v_j$, halved if $E^+ \geq 0.5$. The final score is then

$$s(x) = \sigma\Big(\operatorname{clip}\big(\operatorname{logit} p(x), -4, 4\big) + 5\,(E^+ - E^-)\Big), \tag{5.6}$$

If a message is shaped like a genuine transaction alert or service notice and has no blocking cue, the score is capped at 0.15. A separate "decoy" rule handles the trick of writing "never share your OTP" and then asking for it anyway.

5.4.4 Inference procedure

Algorithm 1 Scoring one message in the live service

Require: message x; frozen vectorizer and model; intent rules

$t \leftarrow \textsc{Tokenize}(\textsc{Clean}(x))$
$p \leftarrow \sigma(\mathbf{w}^\top \phi(t) + b)$
find matched intents I and protective signals P
if a sentence asks for an OTP after a "never share" warning **then**
    add decoy intent
end if
$s \leftarrow \sigma(\text{clip}(\text{logit}\, p, \pm 4) + 5(E^+ - E^-))$
if genuine alert or notice with no blocking cue **then**
    $s \leftarrow \min(s, 0.15)$
end if
return verdict $\mathbf{1}[s \geq 0.5]$, p, s, highlighted phrases, explanation

5.5 Campaign Grouping and Dataset Construction

After links and digits are replaced, a message is reduced to S, the set of its word 3-shingles. Two messages share a campaign if

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|} \;\geq\; 0.6, \tag{5.7}$$

where J is estimated from 96 MinHash permutations with locality-sensitive hashing.

Algorithm 2 outlines how ScamGuard-IN-2026 is assembled and split.

Algorithm 2 Building and splitting ScamGuard-IN-2026

Require: sources R1–R3, synthetic templates S, audit files

normalise anonymised entities in R1; mask personal data
group real messages into campaigns by MinHash ($J \geq 0.6$)
for each reported campaign **do**
    if strong fraud cue and no genuine cue **then**
        label scam
    else
        apply the manual decision (scam, genuine or drop)
    end if
end for
re-read rows that a cross-validated model strongly contradicts
assign each campaign to train, development or test by a salted hash
add synthetic messages to training only; remove exact test duplicates from training
return labelled dataset with split column

5.6 Cross-Domain Study

5.6.1 Domain adaptation

Write A_{train} for the source training split, P for the target adaptation pool and $S_f \subset P$ for a stratified sample of size $|S_f| = \lfloor f\,|P| \rfloor$. The adapted model learns from

$$D_f = A_{train} \cup S_f, \qquad f \in \{0, 0.1, 0.2, 0.4\}, \tag{5.8}$$

and the vectorizer is fitted again on D_f, which lets Indian words into the vocabulary. The pool has $|P| = 1{,}435$ messages, so the three samples hold 143, 287 and 574. All values of f are scored on one fixed target test set, B_{test}, of 615 messages.

5.6.2 Experiment protocol

The protocol of the cross-domain study is written out in Algorithm 3. Every decision is made on source data, and the final target test set is touched only to score models.

Algorithm 3 Cross-domain evaluation and lightweight domain adaptation

Require: cleaned source set A (UCI), cleaned target set B (Indian), seeds $\{42, 123, 2026\}$, fractions $\{0, 0.1, 0.2, 0.4\}$

choose α (Naive Bayes) and C (Logistic Regression) by 5-fold cross-validation on A_{train}
for each seed **do**
    split A 80/20 into A_{train}, A_{test} and B 70/30 into pool P and test B_{test} (stratified)
    for each fraction f **do**
        $S_f \leftarrow$ stratified sample of $f \cdot |P|$ messages from P
        fit TF-IDF and the classifier on $A_{train} \cup S_f$
        score A_{test} and B_{test}; store predictions
    end for
end for
report mean ± SD over seeds; McNemar test of each adapted model against $f = 0$

5.7 Back End, Database and Front End

The FastAPI back end exposes REST routes for sign-in, scans, history, cases, analytics, the copilot, the safety tools and the cross-domain study. The Next.js front end calls them through its own origin and offers the quick check, full analysis, dashboard, safety tools and cross-domain page in English and Hindi. The main tables of the PostgreSQL database are listed in Table 5.4.

Table 5.4: Main database tables

Table	Contents
users	Account, hashed password, role, preferences
refresh_tokens	Issued refresh tokens for rotation and revocation
messages	Each analysed input with its type (text, URL, e-mail, image, PDF, QR)
predictions	Verdict, classifier probability, risk score, category, explanation, model version
cases, case_scan_links, case_timeline_entries	Investigation cases, the scans linked to them and their notes
community_reports	One report per user for a number, link domain, UPI ID or e-mail
alerts, audit_log, model_registry_meta	Alerts, security audit trail and registered model versions

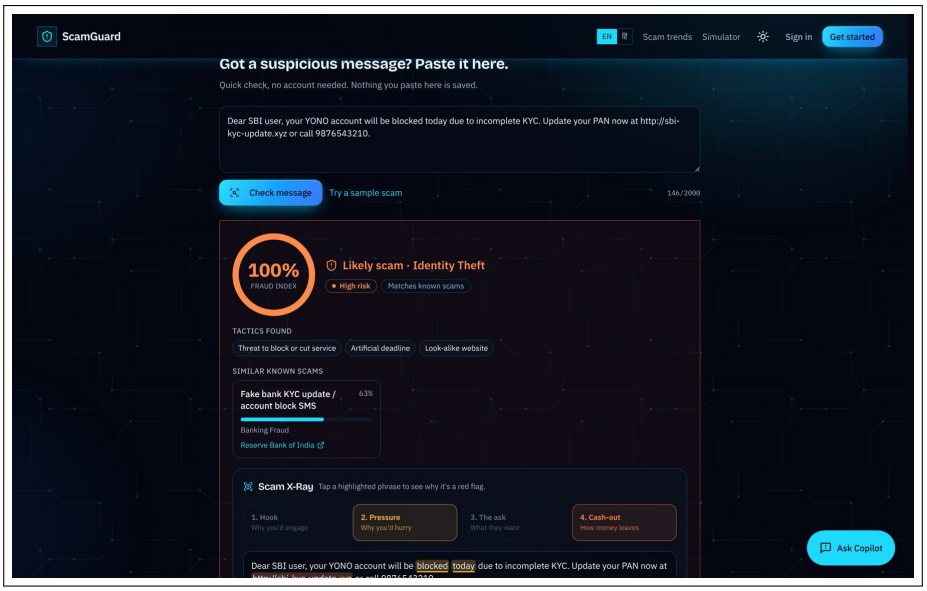


Figure 5.5: Quick check of a fake KYC message: verdict, matched tactics and Scam X-Ray

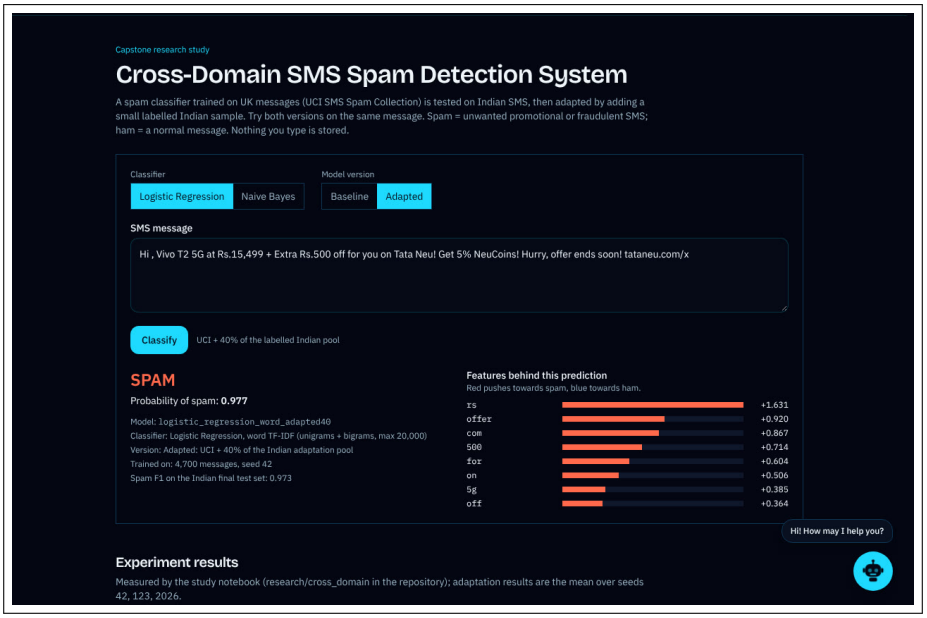


Figure 5.6: Cross-domain page: the adapted model flags an Indian promotion that the baseline misses

6 Benchmarking and Results

6.1 Evaluation Objective and Datasets

For the main detector we picked models on the development split and then retrained the chosen one on training and development data together, 9,513 messages in all. The numbers below come from the real test split: 1,874 messages (665 genuine, 1,209 scams) belonging to campaigns the model never saw. While designing the rules, the team also wrote 259 challenge messages. Because we wrote them ourselves, we treat them as a development check and report them separately rather than as an independent test. In the cross-domain study every setting was tuned by cross-validation on the UCI training split, every adapted model faced the same untouched Indian test set, and the whole study was run three times with seeds 42, 123 and 2026. All experiments ran on a two-core CPU with Python 3.13 and scikit-learn 1.7.2.

Testing of the software happened at three levels. First, 836 automated back-end and ML tests cover preprocessing, features, the model registry, the detector, the rules, the cross-domain service and every API route; one of them confirms that the models served by the website give exactly the notebook's scores. Another 14 tests cover the front end, and all of them pass. Second, continuous integration runs the suite on every change before it is deployed. Third, the live site is checked after each deployment.

6.2 Metrics

Scam (or spam) is the positive class throughout:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{6.1}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{6.2}$$

$$\text{False-alarm rate} = \frac{FP}{FP + TN}. \tag{6.3}$$

We draw confidence intervals from 1,000 bootstrap resamples, and when an experiment is repeated over seeds we give the mean ± standard deviation.

When two models are scored on the same messages, only the messages they disagree on carry information. Let b count those that only the first model gets right and c those that only the second gets right. If the two models were equally good, $\min(b, c)$ would follow a

binomial distribution with $n = b + c$ and probability 0.5. The exact McNemar test [24, 25] turns this into a two-sided p-value.

6.3 Aggregate Results

Table 6.1: Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate)

System	Acc.	Prec.	Rec.	F1	FAR	AUC
Classifier only	97.5	98.6	97.6	98.1	2.6	0.997
Deployed system (hand rules)	97.1	98.2	97.2	97.7	3.2	0.995
Learned fusion	**97.8**	**98.7**	**97.8**	**98.3**	**2.3**	0.997

Out of 1,209 test scams the deployed system stops 1,175, and it lets 644 of the 665 genuine messages through. Its accuracy has a 95% confidence interval of 96.3–97.8%, and its false-alarm rate one of 2.0–4.6%. On the twelve messages taken from 2025–26 advisories it flags 10 of the 11 scams and leaves the one genuine notice alone, though twelve messages are far too few to conclude much.

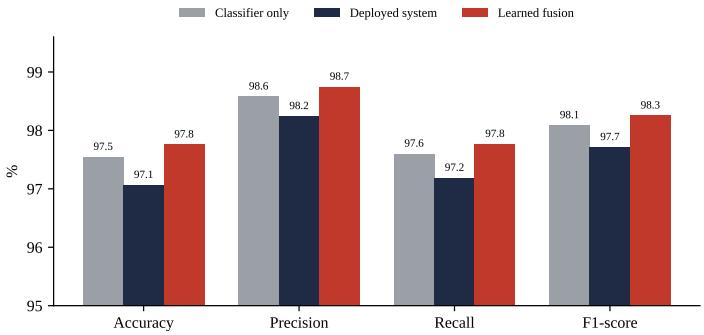


Figure 6.1: Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%)

Models and features. All classifiers in Table 6.2 share one split. Among the strong ones, logistic regression on word plus character features raises the fewest false alarms. We also tried two deep models under the same protocol. A character CNN scored 97.3% F1 with 4.4% false alarms and a word BiLSTM 97.4% with 5.6%, while logistic regression scored 97.8% and 3.0% in that run. Neither gap is statistically significant (McNemar $p = 0.09$ and 0.11), so the extra complexity bought nothing.

Table 6.2: Classifiers on the real test set (%); "Challenge" is F1 on the 259 team-written messages

Model	Features	F1	False alarms	Challenge F1
Naive Bayes	word	96.2	6.2	88.0
Naive Bayes	word + char	95.5	6.5	86.6
Logistic regression	word	98.0	2.7	89.9
Logistic regression	char	96.9	4.8	86.8
Logistic regression	**word + char**	**98.1**	**2.6**	90.2
Linear SVM	word	97.7	5.6	91.6
Linear SVM	word + char	98.2	3.9	92.0
Random forest	word	97.5	2.7	73.7
Random forest	word + char	97.6	3.3	73.9

Label audit. Suppose we skip the audit and call every reported message a scam. The same model then reaches 99.4% recall, but it flags 34.0% of genuine messages. With the audited labels, false alarms fall to 2.6% and recall is still 97.6%. The two models disagree on 233 test messages, and the audited one is right on 210 of them (McNemar $p < 10^{-30}$, Figure 6.2).

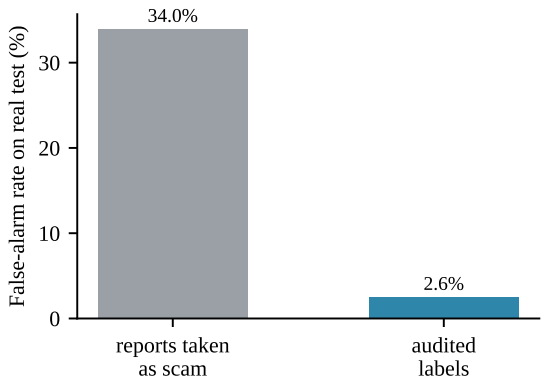


Figure 6.2: False-alarm rate with report labels and with audited labels

Split and synthetic data. When the real data is split at random, 793 test messages have a near-copy in training. That split reports 98.3% F1 and 0.8% false alarms; the campaign-aware split reports 97.7% and 1.4%. In other words, the random split hides close to half of the false alarms. Table 6.3 changes what the model is trained on. Synthetic messages on their own are a poor stand-in for real Indian SMS. Real messages on their own catch just 35.6% of the scams in the team-written challenge set. Using both keeps real-data F1 at 98.1% and lifts challenge recall to 82.2%; the price is more false alarms on real data, 2.6%

instead of 1.4%.

Table 6.3: Effect of the training data (logistic regression, word + char, %)

Training data	Real F1	Real FAR	Challenge recall	Challenge F1
Synthetic only	87.4	31.1	86.4	90.3
Real only	97.7	1.4	35.6	51.2
Real + synthetic	98.1	2.6	82.2	90.2

Intent rules. The result here surprised us. On real messages from unseen campaigns, the hand-weighted rules make things slightly worse: F1 goes from 98.1% to 97.7% and false alarms from 2.6% to 3.2%. On our own challenge messages the same rules look excellent, lifting F1 from 90.2% to 95.6%. Weights learned from data beat the hand-set ones on real messages ($p = 0.002$). We still keep the rules, but for what they actually give: phrase-level explanations and some cover for scam types missing from the real training data. They are not an accuracy gain.

Operating point and errors. The deployed score reaches a ROC-AUC of 0.995 (Figures 6.3 and 6.4). At the default threshold it misses 34 scams and wrongly flags 21 genuine messages. Fifteen of those 21 are real bank messages that users had reported and that do ask the reader to click a link. Moving the threshold to 0.9 cuts false alarms to 1.1% while recall drops to 92.7%. For banking scams alone, recall is 97.7% (895 of 916). Model and vectorizer together take 1.6 MB, training finishes in 2.7 seconds, and one message is scored in about 2 ms, or 4 ms with the rules.

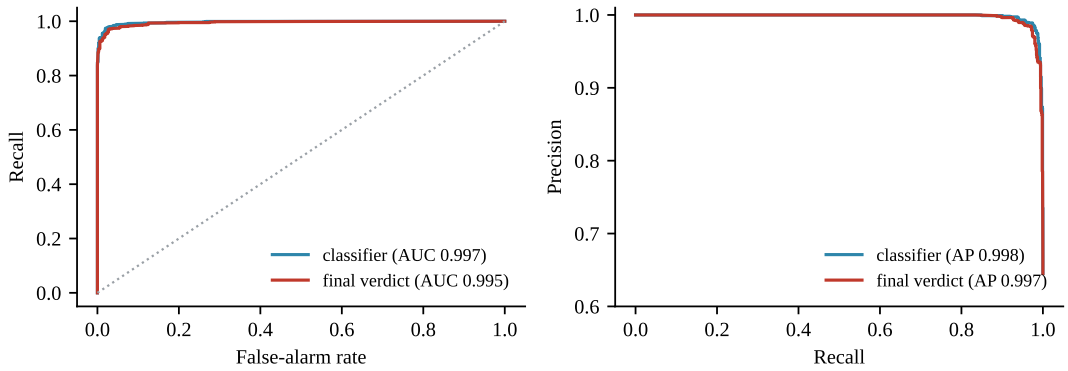


Figure 6.3: ROC and precision–recall curves on the real test set

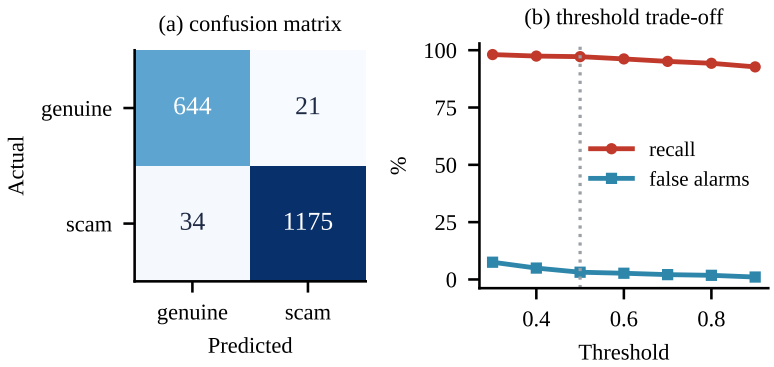


Figure 6.4: (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold

6.4 Cross-Domain Generalization and Adaptation

6.4.1 Cross-domain generalization (RQ1 and RQ2)

For Table 6.4 we took the models trained only on UCI data (seed 42) and ran them, unchanged, on all 2,050 cleaned Indian messages. Each one loses between 0.23 and 0.36 F1. The failure has a clear shape: precision is still high at 0.88–0.96, but recall sinks to 0.40–0.55. The models seldom raise a false alarm; they simply do not recognise about half of the Indian spam. Logistic Regression on word features holds up best.

Table 6.4: Trained on Dataset A only (seed 42): spam F1 on the UCI test split and on all of Dataset B

Model	Feat.	F1 UCI	F1 India	Prec. India	Rec. India
Naive Bayes	word	0.906	0.551	0.883	0.401
Logistic Regression	word	0.911	**0.678**	0.931	0.533
Linear SVM	word	0.914	0.615	0.949	0.455
Naive Bayes	char	0.948	0.684	0.908	0.549
Logistic Regression	char	0.940	0.638	0.942	0.482
Linear SVM	char	0.943	0.578	0.956	0.414

For Naive Bayes the smoothing constant decides almost everything. With the library default, $\alpha = 1$, it scores 0.726 F1 on UCI and just 0.014 on Indian messages, because it calls nearly every Indian message ham. With α tuned on source data only, the same model scores 0.906 and 0.551.

Why the domains differ. About one word in five (20.7%) of a typical Indian message never occurs in the UCI training data; for held-out UCI messages the figure is 7.8%. Indian messages are also longer, with a median of 81 characters against 59. Links tell the same story: 87.0% of Indian spam carries a URL, compared with 1.6% of UCI test messages. And 24.3% of Indian spam mentions money as "Rs" or "INR", which a UK-trained model has barely seen.

6.4.2 Lightweight domain adaptation (RQ4)

Table 6.5 and Figure 6.5 show what labelled Indian messages do. The first 10% of the pool does most of the work. Those 143 messages take the mean Indian F1 of Logistic Regression from 0.681 to 0.964 and that of Naive Bayes from 0.582 to 0.950, while F1 on UCI stays around 0.93. Every model, feature type and fraction improved in all three seeds, and McNemar's test gave $p < 0.05$ in each of the 36 comparisons. Most of the improvement comes from spam that is now caught. For Logistic Regression at seed 42, missed spam falls from 96 messages to 3, and false alarms barely move, from 10 to 9 (Figure 6.6).

Table 6.5: Spam F1 on the final Indian test set (615 messages), word TF-IDF, mean ± SD over three seeds

Indian data added	Naive Bayes	Logistic Regression
0% (source-only)	0.582 ± 0.018	0.681 ± 0.025
10% (143 messages)	0.950 ± 0.006	0.964 ± 0.004
20% (287 messages)	0.967 ± 0.007	0.968 ± 0.005
40% (574 messages)	**0.978** ± 0.009	**0.975** ± 0.005
100% (reference only)	0.987 ± 0.007	0.987 ± 0.003

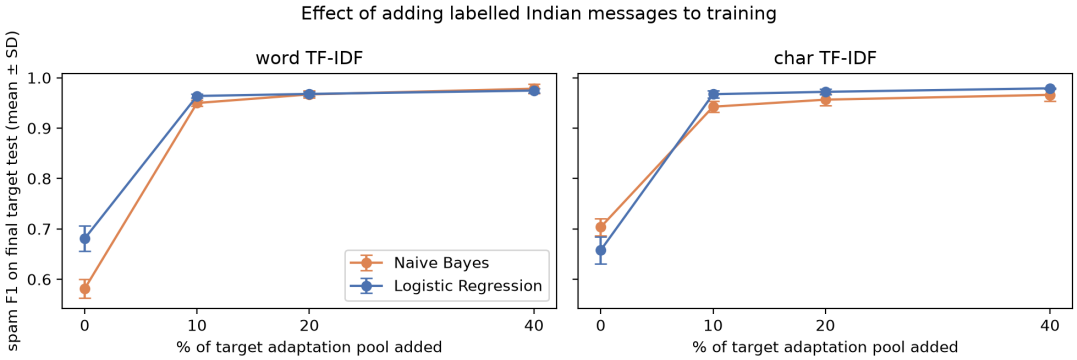


Figure 6.5: Spam F1 on the final Indian test set as labelled Indian messages are added (mean ± SD over three seeds)

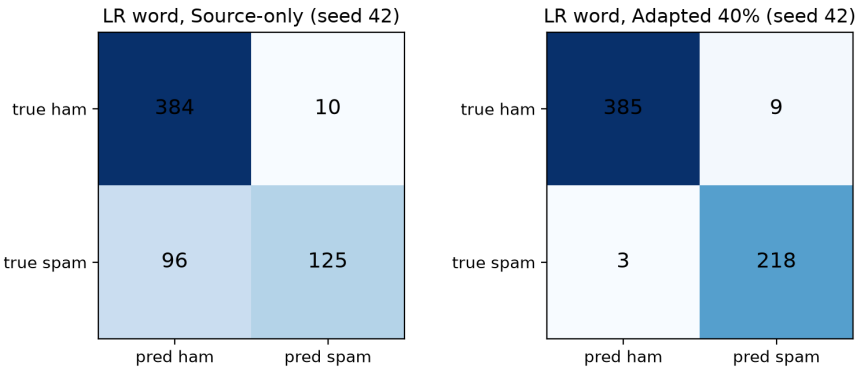


Figure 6.6: Logistic Regression (word TF-IDF, seed 42) on the Indian test set, before and after adding 40% of the pool

Near-duplicate check. Indian promotions are sent from templates, so a random split may put a near-copy of a test message into the adaptation pool, and the model could then score well just by remembering it. To rule this out we grouped Dataset B with MinHash (Jaccard ≥ 0.6) and got 1,811 groups from 2,050 messages. Under the random split, between 86 and 104 of the 615 test messages had a near-copy in the pool, depending on the seed. When we kept whole groups on one side and ran the adaptation again, the gains shrank only a little (Table 6.6). Memorised templates do not explain the effect.

Table 6.6: Target F1 with a random and a group-aware split of Dataset B (word TF-IDF, mean ± SD)

Model	Added	Random split	Group-aware split
Logistic Regression	0%	0.681 ± 0.025	0.652 ± 0.062
Logistic Regression	10%	0.964 ± 0.004	0.952 ± 0.008
Logistic Regression	40%	0.975 ± 0.005	0.969 ± 0.011
Naive Bayes	0%	0.582 ± 0.018	0.551 ± 0.017
Naive Bayes	10%	0.950 ± 0.006	0.937 ± 0.015
Naive Bayes	40%	0.978 ± 0.009	0.960 ± 0.015

6.4.3 Word versus character features (RQ3)

We expected character n-grams to cope better with unfamiliar spellings, and that was only half true (Table 6.7, Figure 6.7). Without adaptation they lifted Indian F1 for Naive Bayes but lowered it a little for Logistic Regression. Once Indian data is added, the two feature types end up within 0.01 of each other for Logistic Regression. So neither representation wins for both classifiers.

Table 6.7: Source-only models: word versus character TF-IDF (mean ± SD over three seeds)

Model	Features	Source F1	Target F1	Target recall
Naive Bayes	word	0.927	0.582 ± 0.018	0.430
Naive Bayes	character	0.941	0.704 ± 0.017	0.581
Logistic Regression	word	0.928	0.681 ± 0.025	0.535
Logistic Regression	character	0.942	0.658 ± 0.027	0.502

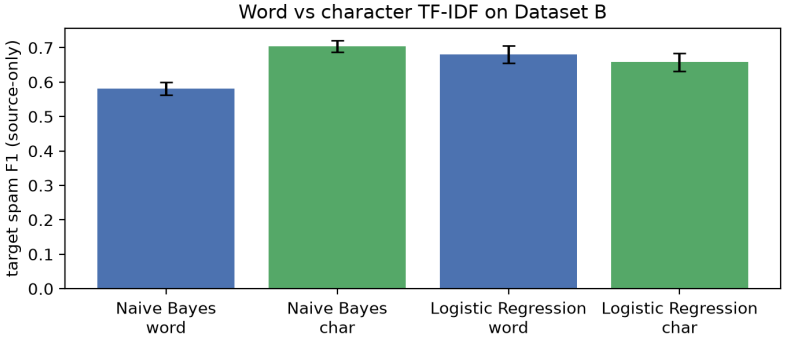


Figure 6.7: Target F1 of the source-only models with word and character TF-IDF

6.4.4 Error analysis (RQ5)

We read every mistake Logistic Regression (word TF-IDF, seed 42) made on the Indian test set, 106 before adaptation and 12 after, and sorted each one into a category by hand (Table 6.8, Figure 6.8). Before adaptation, the bulk of the errors are Indian promotions full of words the UCI model had never met. After adaptation, eight of the 12 remaining errors are tiny chat messages that give the model almost nothing to work with. We also noticed nine messages in Dataset B whose labels look doubtful to us.

Table 6.8: Error categories on the Indian test set (Logistic Regression, word TF-IDF, seed 42)

Category	Source-only	Adapted (40%)
Indian brand or telecom promotion with unfamiliar vocabulary	61	0
Operator service notice (data alert, pack expiry, missed call)	14	0
Questionable label: safety advice or receipt labelled spam	9	1
Very short or context-free chat (ham)	8	8
Fraud with disguised spelling (“t0”, “n0w”)	6	2
Loan or debt message	3	0
Market or work commentary with spam-like words (ham)	2	1
Code-mixed Hinglish promotion	2	0
Lost characters (“????”)	1	0
Total	**106**	**12**

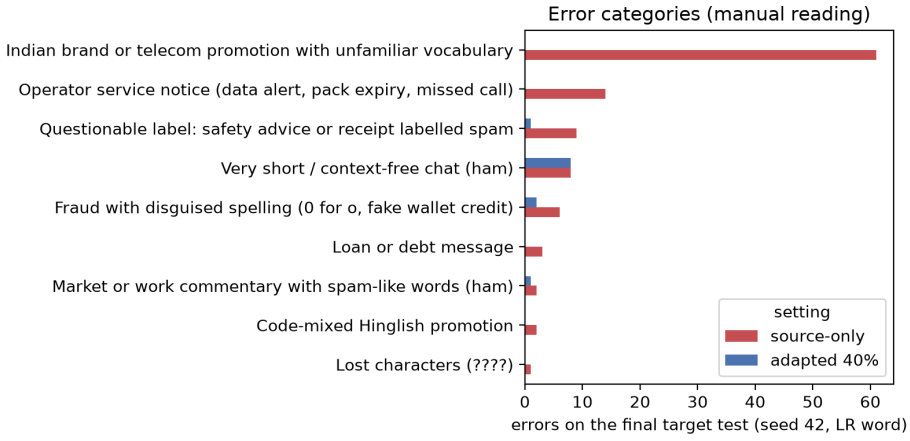


Figure 6.8: Error categories before and after adaptation

6.5 Interpretation Boundaries and Threats to Validity

The results should be read with the following limits in mind.

- **Dates of the data.** Most reported messages in ScamGuard-IN-2026 date from 2022–2024. Scam types from 2025–26 enter training only through synthetic messages and enter testing only through 12 advisory messages.
- **How reliable the labels are.** A single annotator made the manual audit decisions, so we cannot report agreement between annotators. The Indian dataset used in the cross-domain study does not say how it was labelled, and nine of its labels look doubtful.
- **Only two domains.** With one source and one target dataset, the study cannot claim that the findings hold for every pair of domains. Three seeds also give only a rough idea of the spread.
- **An easy cue in Dataset B.** Not one ham message in it has a URL, whereas 87% of its spam does. An adapted model can lean on that cue, and real messages will not always be so tidy.
- **Disguised spellings.** The main detector's cleaner strips dotted and digit-swapped spellings before building features. The intent rules catch many such messages, but we have not tested attackers who adapt to the system.
- **Scope.** The system judges single messages in English and Hindi only, and in the cross-domain study "spam" also covers legitimate promotions.

The assumptions behind the study and the main risks are summarised below. The audited labels are taken to be right, and where we could not tell we dropped the message instead of guessing. Each public dataset is taken to be a fair enough sample of its domain for a controlled comparison, and the final target test set is kept out of training, feature fitting and every choice of settings. The free-tier hosting allows 512 MB of memory, so large transformer models cannot be served in production.

Table 6.9: Risks and how they are handled

Risk	**Mitigation**
Reported data carries wrong labels	Audit by rule and by hand, then a second pass on cases a cross-validated model disputes
Near-duplicates make scores look better than they are	Split by campaign; group-aware split check in the cross-domain study
New scam types are missing from training data	Synthetic training messages, intent rules, and a small labelled local sample (adaptation)
Disguised spellings (K.Y.C, bl0cked)	Caught by the intent rules; normalising them before features are built is left for future work
The free-tier server falls asleep	Keep-alive pings, front end hosted on Vercel, automatic retries
The outside AI service is slow or down	Time limits and fallback answers; the detector itself never depends on it

6.6 Application-Level Evaluation

Every functional requirement listed in Section 3.1 is available in the deployed application. On the cross-domain page a user can run one message through both the baseline and the adapted model and compare them. For example, an Indian retail offer gets 0.149 (ham) from the baseline Logistic Regression and 0.977 (spam) from the adapted one. Screenshots of the quick check and the cross-domain page are in Chapter 5.

7 Conclusion and Future Enhancements

7.1 Conclusion

ScamGuard spots scam and spam SMS aimed at Indian users and tells them why. Trained on our audited ScamGuard-IN-2026 corpus and tested with a campaign-aware split, the deployed detector gets 97.1% of real messages from unseen campaigns right with 3.2% false alarms. It replies within milliseconds and shows the exact phrases behind every verdict.

The cross-domain study that our guide proposed adds a second lesson. A classifier of the same kind, trained on another country's messages, loses about a third of its F1 on Indian SMS, and a small, carefully labelled local sample is enough to win it back. Anyone putting such a filter into service should test it on local data first and invest in a few hundred verified local labels before turning to a heavier model.

All of this runs in a live application: quick check, full analysis, Hindi support, safety tools, a cross-domain page and a Streamlit demo. Every number in this report can be regenerated from the project repository.

Three findings matter most. The labels mattered more than the model. Report labels led to 34.0% false alarms and audited labels to 2.6%, whereas all sensible classifiers, the CNN and BiLSTM included, landed within a few points of one another. The way we evaluate also changed the conclusions. A campaign-aware split reveals errors that a random split conceals, and rules that shine on their authors' own messages add nothing on independent data. Finally, a standard spam classifier does not travel well between countries. Trained on UK messages, it misses roughly half of Indian spam. Yet 143 labelled Indian messages bring F1 back above 0.95 without hurting UK performance, in every seed and with significance under McNemar's test.

7.2 Future Enhancements

The work can be extended in the following directions.

- Finish collecting the team's 2026 test set and run both evaluations again on it.
- Bring in a second annotator for the label audit and report how often the two agree.
- Widen the cross-domain study to more target datasets and seeds, and try adaptation with less than 10% of the pool.

- Undo disguised spellings before features are built, and test against attackers who adapt or paraphrase.
- Cover Kannada, Tamil, Telugu and other Indian languages, and compare small transformer models under the same protocols.

Summary

Major Project Phase 2 builds ScamGuard, a system that reads one SMS or other message, decides whether it is a scam or genuine, explains the decision and suggests the next step. The completed work includes the ScamGuard-IN-2026 dataset of 11,399 messages with an audit of its labels, campaign-aware splitting with MinHash, the TF-IDF and logistic-regression detector joined with 31 intent patterns, the FastAPI back end with its PostgreSQL database, the Next.js web application in English and Hindi, and the cross-domain generalization and adaptation study proposed by our guide. On 1,874 real messages from campaigns the model never saw, the deployed system scores 97.1% accuracy with 3.2% false alarms. In the cross-domain study, adding 10% of a labelled Indian pool (143 messages) raised the mean Indian F1 of Logistic Regression from 0.681 to 0.964. These are results on public datasets that we audited ourselves; they do not claim the same accuracy on all future scam messages.

Key deliverables achieved include:

Dataset and Label Audit: ScamGuard-IN-2026 was assembled from public reports, real Indian SMS, advisory texts and synthetic training messages. An audit of 2,922 decisions showed that 19.5% of the retained reports were genuine, and the corrected labels cut false alarms from 34.0% to 2.6% for the classifier alone.

Detection Engine: The classifier uses 35,148 word and character TF-IDF features, and the intent evidence and ten protective signals are fused with a capped adjustment so that each verdict can be explained in plain words.

Cross-Domain Study: Two models trained on the UCI SMS Spam Collection were tested unchanged on an Indian SMS dataset. Three seeds and McNemar's test (all 36 comparisons significant at $p < 0.05$) show that a small amount of local data repairs most of the drop.

Application and Deployment: The Next.js front end runs on Vercel, the FastAPI back end on Render and the database on Neon, with 836 back-end and 14 front-end automated tests, a cross-domain page and a Streamlit demo.

References

[1] S. Agarwal, A. Papasavva, G. Suarez-Tangil, and M. Vasek, "Fishing for smishing: Understanding SMS phishing infrastructure and strategies by mining public user reports," in *Proc. ACM Internet Measurement Conference (IMC '25)*, Madison, WI, USA, 2025, doi: 10.1145/3730567.3764431.

[2] S. Agarwal, G. Suarez-Tangil, and M. Vasek, "An overview of 7726 user reports: Uncovering SMS scams and scammer strategies," in *Proc. Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2026.

[3] M. M. A. Pritom, S. M. Sanjari, M. Mia, A. M. Shibli, S. M. M. Hossain, M. Ismail, and S. Xu, "Short message service (SMS) phishing attacks and defenses: A systematic review," arXiv:2604.11429, 2026.

[4] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symposium on Document Engineering*, Mountain View, CA, USA, 2011, pp. 259–262.

[5] T. A. Almeida, J. M. Gómez Hidalgo, and T. P. Silva, "Towards SMS spam filtering: Results under a new dataset," *International Journal of Information Security Science*, vol. 2, no. 1, pp. 1–18, 2013.

[6] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep learning to filter SMS spam," *Future Generation Computer Systems*, vol. 102, pp. 524–533, 2020.

[7] S. Mishra and D. Soni, "Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis," *Future Generation Computer Systems*, 2020.

[8] G. Sonowal and K. S. Kuppusamy, "SmiDCA: An anti-smishing model with machine learning approach," *The Computer Journal*, vol. 61, no. 8, pp. 1143–1157, 2018, doi: 10.1093/comjnl/bxy039.

[9] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "Machine learning driven smishing detection framework for mobile security," arXiv:2412.09641, 2024.

[10] D. Timko and M. L. Rahman, "Smishing Dataset I: Phishing SMS dataset from Smishtank.com," in *Proc. 14th ACM Conference on Data and Application Security and Privacy (CODASPY '24)*, Porto, Portugal, 2024, doi: 10.1145/3626232.3653282.

[11] F. D. M. A. Ali, S. M. Saide, R. Sousa-Silva, and H. Lopes Cardoso, "MOZ-Smishing: A benchmark dataset for detecting mobile money frauds," in *Proc. 6th Workshop on African Natural Language Processing*, Vienna, Austria, 2025, pp. 158–166, doi: 10.18653/v1/2025.africanlp-1.23.

[12] M. Labonne and S. Moran, "Spam-T5: Benchmarking large language models for few-shot email spam detection," arXiv:2304.01238, 2023.

[13] M. Ahmadi, M. Khajavi, A. Varmaghani, A. Ala, K. Danesh, and D. Javaheri, "Leveraging large language models for cybersecurity: Enhancing SMS spam detection with robust and context-aware text classification," arXiv:2502.11014, 2025.

[14] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021, doi: 10.1613/jair.1.12125.

[15] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. 60th Annual Meeting of the ACL*, Dublin, Ireland, 2022, pp. 8424–8445, doi: 10.18653/v1/2022.acl-long.577.

[16] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 2023, doi: 10.1016/j.patter.2023.100804.

[17] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of SEQUENCES 1997*, Positano, Italy, 1997, doi: 10.1109/SEQUEN.1997.666900.

[18] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] JuniorAlive, "India Spam SMS Classification," GitHub repository, 2024. [Online]. Available: https://github.com/junioralive/india-spam-sms-classification

[21] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Machine Learning*, vol. 79, pp. 151–175, 2010, doi: 10.1007/s10994-009-5152-4.

[22] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

[23] H. Daumé III, "Frustratingly easy domain adaptation," in *Proc. 45th Annual Meeting of the Association for Computational Linguistics*, Prague, Czech Republic, 2007, pp. 256–263.

[24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[25] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, doi: 10.1162/089976698300017197.

SDG Analysis

The ScamGuard project aligns with the following United Nations Sustainable Development Goals (SDGs):

SDG 16: Peace, Justice and Strong Institutions (target 16.4, illicit financial flows). Many payment frauds begin with one text message. If a user spots the trap before paying or sharing an OTP, the money never reaches the fraudster. The application also sends users to the official channels: the 1930 helpline and cybercrime.gov.in.

SDG 8: Decent Work and Economic Growth (target 8.10, access to financial services). People adopt digital payments only when they feel safe using them. Telling real bank and UPI alerts from imitations helps keep that confidence.

SDG 9: Industry, Innovation and Infrastructure (target 9.c, access to ICT). The model is small, runs on free cloud servers and replies in milliseconds, so the service costs its users nothing. The cross-domain results add that a model can be moved to a new region with a few hundred labelled local messages.

SDG 10: Reduced Inequalities. Hindi support and a simpler elder mode are meant for the people fraudsters target most and English-only tools serve least.

Table 2: Programme Outcomes Mapping with ScamGuard and Domain Adaptation

PROGRAMME OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PO1	3	Machine learning, NLP and statistics are used both to build the scam classifier and to judge how well it works.
PO2	3	We studied why current filters struggle with Indian SMS and traced the problem to unchecked labels, leaky evaluation and domain shift.
PO3	3	A dataset, a detector and a web application were designed to meet targets for accuracy, speed and explanation.
PO4	3	Experiments are controlled: held-out campaigns, a separate target domain, bootstrap intervals, three seeds and significance tests.

PROGRAMME OUTCOMES	Level (1/2/3)	Justification if addressed
PO5	3	Modern tools throughout: scikit-learn, MinHash, FastAPI, Next.js, Streamlit, GitHub Actions and cloud hosting.
PO6	3	Helps people avoid financial fraud and directs them to the official places to report it.
PO7	1	Light enough for free, low-power servers; no GPU is needed.
PO8	3	Personal data is masked, dataset licences are followed and the limitations are stated openly.
PO9	3	Four students shared the work on data, model, back end and front end.
PO10	3	Results reach readers through this report, two research papers, a presentation and an interface that explains itself.
PO11	2	Work was planned in two phases, and hosting stays inside free-tier limits.

Plagiarism Report

DrillBit

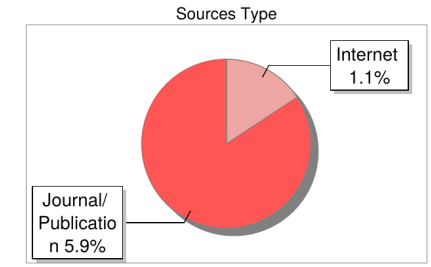
The Report is Generated by DrillBit Plagiarism Detection Software

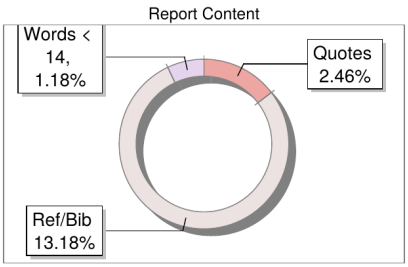
Submission Information

Author Name	Samir Chaudhary
Title	ScamGuard AI
Paper/Submission ID	7337392
Submitted by	library@bmsce.ac.in
Submission Date	2026-10-28 16:13:08
Total Pages, Total Words	52, 9951
Document type	Project Report

Result Information

Similarity **7 %**





Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

AI Content Detection Report
Submission ID: 7337392

AI TEXT DETECTED (%)
Percentage of content showing AI-generated patterns

12.59

Matched 1272 words showing AI-pattern signal

POSITION ON SCALE
Where this score falls on a 0 - 100% range

0% 100%

12.59 %

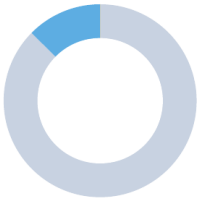
01 Submission Information

Document metadata

Author Name	Samir Chaudhary
Title	ScamGuard AI
Paper/Submission ID	7337392
Submitted By	library@bmsce.ac.in
Submission Date	2026-10-28 16:13:08
Document Type	Project Report
Language	English
Text [Pages, Sentences, Words]	[52, 859, 9951]

02 Content Composition

Colors match in-document highlighting



Human Written — 87.41 %
Original or non-flagged content

AI Text — 12.59 %
Highlighted in blue in the document

Excluded — 0.00 %
Manually excluded content

SCAN

AI INTEGRITY HELP
Guides, policy, and best practices.

▶ **AI Help Centre**
drillbitglobal.com/ai-content-detection

▶ **Detection Methodology**
drillbitglobal.com/ai-method